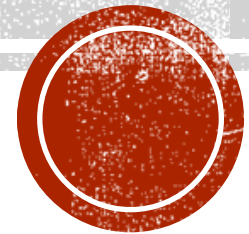


UNIT 1: HADOOP YARN



AGENDA

- What is YARN?
- Why YARN? (The Problem with Hadoop 1.0)
- Key Features of YARN
- YARN Architecture: Core Components
- Application Workflow
- Advantages & Disadvantages
- Summary



WHAT IS YARN?

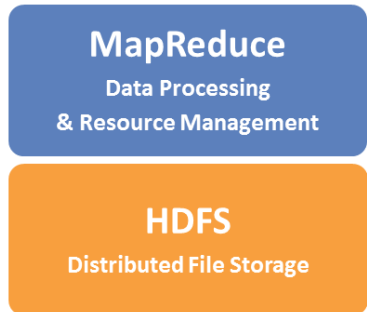
- **YARN** = Yet **A**nother **R**esource **N**egotiator
- Introduced in **Hadoop 2.0** to overcome limitations of Hadoop 1.0.
- Originally a "Redesigned Resource Manager," now evolved into a **large-scale distributed operating system** for Big Data.
- **Key Innovation:** Separates the **resource management** layer from the **data processing** layer.
- Enables multiple data processing engines (batch, interactive, streaming, graph) to run on data stored in HDFS.



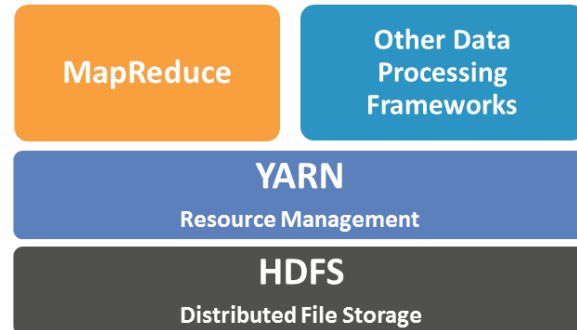
WHY YARN? THE PROBLEM WITH HADOOP 1.0



Hadoop v1.0



Hadoop v2.0



The Job Tracker Bottleneck:

- Single Job Tracker managed both cluster resources (CPU, memory) and application execution (scheduling, monitoring).
- Limited scalability beyond ~4,000 nodes.
- Supported only MapReduce processing.

YARN's Solution:

- Splits the Job Tracker's responsibilities into two daemons:
 - Global Resource Manager
 - Per-Application Application Master
- This separation allows for better scalability, efficiency, and support for diverse workloads.



KEY FEATURES OF YARN

Scalability: The scheduler in Resource manager of YARN architecture allows Hadoop to extend and manage thousands of nodes and clusters.

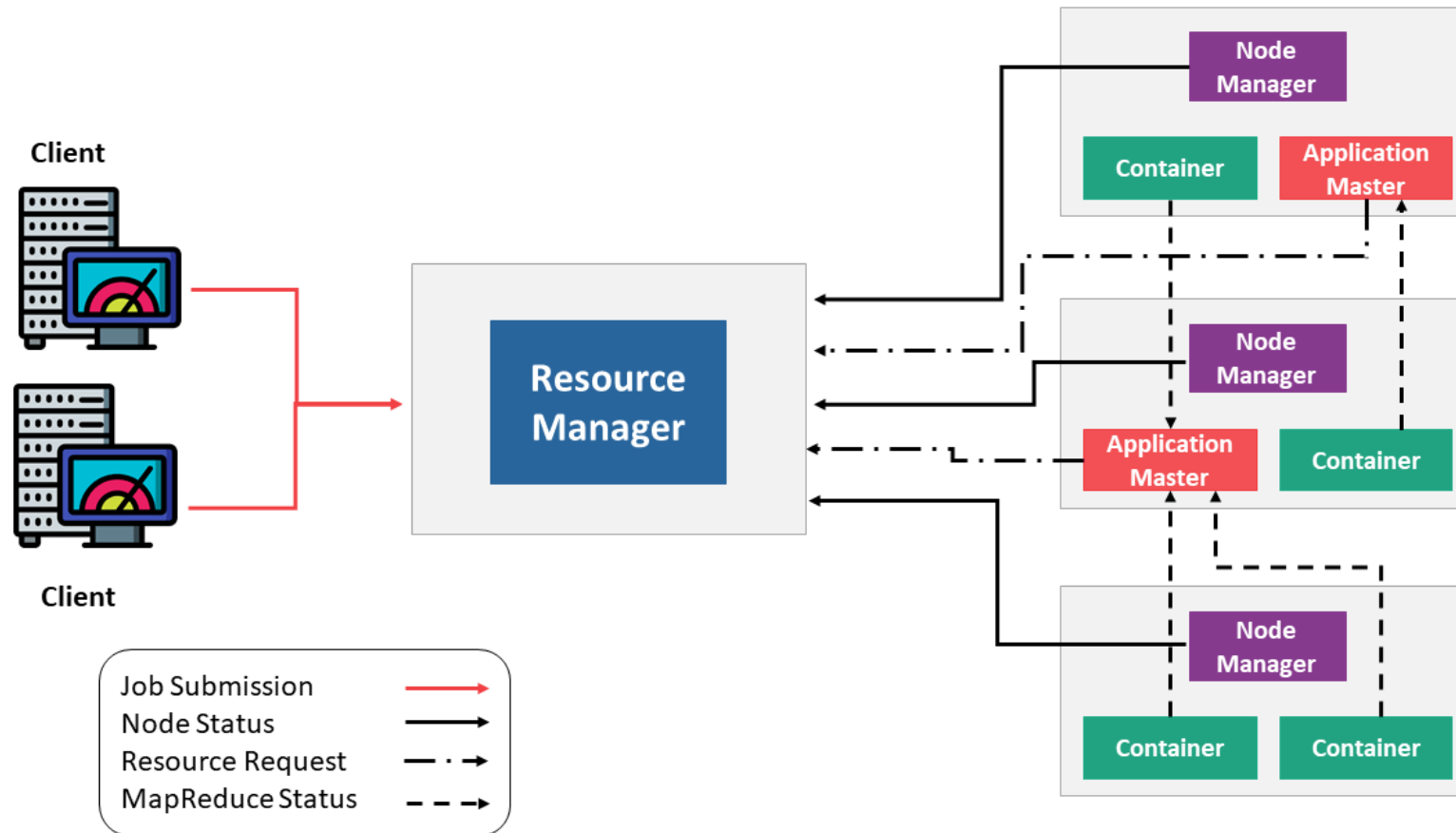
Compatibility: YARN supports the existing map-reduce applications without disruptions thus making it compatible with Hadoop 1.0 as well.

Cluster Utilization: Since YARN supports Dynamic utilization of cluster in Hadoop, which enables optimized Cluster Utilization.

Multi-tenancy: It allows multiple engine access thus giving organizations a benefit of multi-tenancy.



YARN ARCHITECTURE: OVERVIEW



Core Components:

- Client
- Resource Manager
- Node Manager
- Application Master
- Container



COMPONENTS OF YARN ARCHITECTURE: CLIENT

a) Client

- The Client is the entity that initiates and submits the application (such as a MapReduce job) to the YARN framework.
- It communicates with the Resource Manager to request execution, monitors the job status, and can also interact with the Application Master for progress updates.
- It essentially acts as the user's interface to launch and manage applications on the Hadoop cluster.



COMPONENTS OF YARN ARCHITECTURE: **RESOURCE MANAGER**

b) Resource manager

- The "**master**" daemon of YARN.
- Responsible for global resource assignment and management.
- **Two Main Components:**
 - **Scheduler:** A pure scheduler that allocates resources to applications based on capacity, queues, etc. (Plugins: Capacity Scheduler, Fair Scheduler). It does not monitor applications; nor does it restart failed tasks.
 - **Applications Manager:** Accepts job submissions, negotiates the first container for the Application Master(AM), and restarts the AM on failures.



COMPONENTS OF YARN ARCHITECTURE: **NODE MANAGER**

c) Node Manager

- The "**slave**" daemon that runs on **each node** in the cluster.
- Responsible for managing individual nodes.
- **Key Responsibilities:**
 - Registers with the Resource Manager and sends **heartbeats** (node health).
 - Monitors resource usage (CPU, memory) on its node.
 - Creates, monitors, and kills **containers** as directed by the Resource Manager or Application Master.
 - Manages log files.



COMPONENTS OF YARN ARCHITECTURE:

APPLICATION MASTER & CONTAINER

d) Application Master (AM)

- An application is a single job submitted to a framework.
- One AM per application (e.g., per MapReduce job).
- Negotiates resources (containers) from the Resource Manager.
- Works with the Node Manager(s) to launch and monitor the application's tasks in containers.
- Tracks the status and progress of its specific application and reports back to the RM.
- Requests new containers if tasks fail.

e) Container

- It is a collection of physical resources such as RAM, CPU cores and disk on a single node.
- The unit of allocation, granted by the scheduler.
- The containers are invoked by Container Launch Context(CLC) which is a record that contains information such as environment variables, security tokens, dependencies etc.



APPLICATION WORKFLOW IN HADOOP YARN

1. **Application Submission:** A client submits an application (e.g., a MapReduce job, a Spark application) to the YARN Resource Manager (RM). This submission includes the application's code, configuration, and resource requirements.
2. **Application Master Launch:** The Resource Manager allocates a container on a Node Manager (NM) to launch the Application Master (AM) for the submitted application. The AM is responsible for managing the application's execution.
3. **Application Master Registration:** The newly launched Application Master registers itself with the Resource Manager, informing the RM of its presence and readiness to manage the application.
4. **Resource Negotiation:** The Application Master negotiates with the Resource Manager for the necessary containers (resources like CPU, memory) to execute the application's tasks. The RM considers available resources and the application's requirements during this negotiation.
5. **Container Launch Notification:** Once containers are allocated by the Resource Manager, the Application Master notifies the respective Node Managers to launch these containers and execute the application's tasks within them.
6. **Task Execution:** The application's code and tasks are executed within the allocated containers on the Node Managers.
7. **Monitoring and Status Reporting:** The client can monitor the application's status by contacting either the Resource Manager or the Application Master. The Application Master also monitors the progress of tasks and container status, reporting this information to the Resource Manager.
8. **Application Completion and Deregistration:** Upon completion of all tasks and the application's processing, the Application Master unregisters itself with the Resource Manager, releasing the allocated resources.



ADVANTAGES OF YARN

- **Flexibility:** Run Spark, Flink, Storm, and MapReduce on the same cluster.
- **Efficient Resource Management:** Centralized control for optimal allocation of CPU, memory, and disk.
- **High Scalability:** Designed for clusters with thousands of nodes.
- **Improved Performance & Utilization:** Dynamically allocates resources, reducing waste.
- **Security:** Supports Kerberos authentication, SSH, and secure data transmission.



DISADVANTAGES OF YARN

- **Complexity:** Adds a layer of configuration and management overhead.
- **Performance Overhead:** Resource negotiation and scheduling introduce minor latency.
- **Potential Single Point of Failure:** The Resource Manager is critical (though High Availability setups mitigate this). (Standby by Resource Manager after Hadoop 2.4)
- **Limited Non-Java Support:** While supporting multiple engines, native support for other languages can be limited.



SUMMARY

- YARN is the **central operating system** of Hadoop 2.x+ for resource management.
- It enables Hadoop to be a **multi-purpose, scalable data processing platform** beyond just MapReduce.
- Its architecture, based on a central **Resource Manager**, per-node **Node Managers**, and per-application **Application Masters**, provides a robust framework for managing cluster resources efficiently.
- It is the foundation that allows modern data processing frameworks to thrive on Hadoop.

